Summer of Science 2026

Markov Chains
CS26

Dhruv Parsewar

Mentor : Abhineet Majety

Contents

Probability Notes

Random Variables, Expectation, and Key Distributions

1 Random Variables and Expectation

- **Discrete random variables** → probability mass function (pmf):

$$p(x) = P(X = x)$$

- **Continuous random variables** → probability density function (pdf):

$$P(a \leq X \leq b) = \int_a^b f(x)\,dx$$

1.1 Expectation ($E[X]$) → probability weighted average

$$\text{Discrete} \rightarrow E[X] = \sum x_i p_i$$

$$\text{Continuous} \rightarrow E[X] = \int x f(x)\,dx$$

2 properties of expectation:

1. **Linearity:** $E[aX + bY] = aE[X] + bE[Y]$
2. **Indicator trick:** for event A, $E[1_A] = P(A)$
   (random variable when its value is 1 if it happens, otherwise 0)

2 Variance and Key Properties

Variance: $Var(X) = E[X^2] - (E[X])^2$

Derivation:

$$\begin{aligned}
Var(X) &= E[(X-\mu)^2] \quad (\mu \rightarrow \text{mean}) \\
&= E[(X - E[X])^2] \\
&= E[X^2 - 2XE[X] + (E[X])^2] \quad (\text{Note: } E[X] \text{ is constant}) \\
&= E[X^2] - 2E[XE[X]] + (E[X])^2 \\
&= E[X^2] - 2E[X]E[X] + (E[X])^2 \\
&= E[X^2] - (E[X])^2
\end{aligned}$$

Standard Deviation: $\sqrt{Var(X)}$ *(By defn)*

2.1 Properties of Variance

1. **Constant:** $Var(aX+b) = a^2 Var(X)$

$$\begin{aligned} Var(aX+b) &= E[((aX+b) - E[aX+b])^2] \quad \textit{(By defn)} \\ &= E[((aX+b) - (aE[X]+b))^2] \\ &= E[(aX - aE[X])^2] \\ &= E[a^2(X - E[X])^2] \\ &= a^2 Var(X) \end{aligned}$$

2. **Sum:** $Var(X+Y) = Var(X) + Var(Y) + 2Cov(X,Y)$

$$\begin{aligned} Var(X+Y) &= E[((X+Y) - (E[X]+E[Y]))^2] \\ &= E[((X - E[X]) + (Y - E[Y]))^2] \\ &= Var(X) + Var(Y) + 2E[XY + E[X]E[Y] - XE[Y] - YE[X]] \\ &= Var(X) + Var(Y) + 2\underbrace{(E[XY] - E[X]E[Y])}_{Cov(X,Y) \text{ (By defn)}} \\ &= Var(X) + Var(Y) + 2Cov(X,Y) \end{aligned}$$

3. **If X and Y are independent:**

$$Var(X+Y) = Var(X) + Var(Y) \quad \text{i.e. } Cov(X,Y) = 0$$

coz for independent X and Y, $E[XY] = E[X]E[Y]$

3 Some Common Probability Distributions

3.1 Discrete Distributions

3.1.1 Bernoulli

pmf $\rightarrow P(X=1) = p$ (success), $P(X=0) = 1-p$

- **mean** $\rightarrow E[X] = p$

$$\begin{aligned} E[X] &= \sum x \cdot p(x) \quad \textit{(coz discrete)} \\ &= 1 \cdot p + 0 \cdot (1-p) \\ &= p \end{aligned}$$

- **variance** $\rightarrow Var(X) = p(1-p)$

$$\begin{aligned} Var(X) &= E[X^2] - (E[X])^2 \\ &= (1^2 \cdot p + 0^2 \cdot (1-p)) - p^2 \\ &= p - p^2 \\ &= p(1-p) \end{aligned}$$

3.1.2 Binomial

pmf $\rightarrow P(X = k) = \binom{n}{k} p^k (1-p)^{n-k}$ *(n → total trials, k → success)*

- **mean** $\rightarrow E[X] = np$

$$\begin{aligned} E[X] &= \sum_{k=0}^{n} k \cdot \binom{n}{k} p^k (1-p)^{n-k} \\ &= \sum_{k=1}^{n} n \binom{n-1}{k-1} p^k (1-p)^{n-k} \\ &= np \sum_{k=1}^{n} \binom{n-1}{k-1} p^{k-1} (1-p)^{n-k} \\ &= np(p + (1-p))^{n-1} \\ &= np \end{aligned}$$

- **variance** $\rightarrow Var(X) = np(1-p)$
  Using $E[X^2] - (E[X])^2$, we calculate $E[X(X-1)] + E[X]$:

$$\begin{aligned} E[X(X-1)] &= \sum_{k=0}^{n} k(k-1) \binom{n}{k} p^k (1-p)^{n-k} \\ &= \sum_{k=2}^{n} n(n-1) \binom{n-2}{k-2} p^k (1-p)^{n-k} \\ &= n(n-1)p^2 \sum_{k=2}^{n} \binom{n-2}{k-2} p^{k-2} (1-p)^{n-k} \\ &= n(n-1)p^2 (p + 1 - p)^{n-2} \\ &= n(n-1)p^2 \end{aligned}$$

  So, $Var(X) = n(n-1)p^2 + np - (np)^2 = np(1-p)$

OR Use indicator trick!!!
$X = I_1 + I_2 + \cdots + I_n$ where I_i represents event for i^{th} trial.

$$E[X] = \sum_{i=1}^{n} E[I_i] = np \quad (coz\ E[I_i] = p\ for\ each\ 1 \leq i \leq n)$$

$$Var(X) = \sum_{i=1}^{n} Var(I_i) = np(1-p) \quad (coz\ Var(I_i) = p(1-p)\ for\ each\ 1 \leq i \leq n)$$

3.1.3 Poisson

Binomial to Poisson $\rightarrow$ imagine taking n trials and multiplying them toward infinity, while prob. p of each trial shrinks towards zero, such that avg rate of success $(n \times p)$ remains const. $\rightarrow$ this constant is λ (rate).

pmf $\rightarrow P(X = k) = \frac{e^{-\lambda}\lambda^k}{k!}$ for $k \geq 0$

Binomial dist. $\rightarrow$ Binomial thm. Poisson dist. $\rightarrow$ Taylor series (expansion of e^x): $e^x = \sum_{k=0}^{\infty} \frac{x^k}{k!}$

- **mean** $\rightarrow E[X] = \lambda$

$$E[X] = \sum_{k=0}^{\infty} k \cdot \frac{e^{-\lambda}\lambda^k}{k!} = \sum_{k=1}^{\infty} \frac{e^{-\lambda}\lambda^k}{(k-1)!} = \lambda e^{-\lambda} \sum_{k=1}^{\infty} \frac{\lambda^{k-1}}{(k-1)!} = \lambda e^{-\lambda} e^{\lambda} = \lambda$$

- **variance** $\rightarrow Var(X) = \lambda$

$$Var(X) = E[X^2] - (E[X])^2 \quad (E[X^2]\ computed\ using\ E[X(X-1)] + E[X])$$

$$E[X(X-1)] = \sum_{k=0}^{\infty} k(k-1)\frac{e^{-\lambda}\lambda^k}{k!} = \lambda^2 e^{-\lambda} \sum_{k=2}^{\infty} \frac{\lambda^{k-2}}{(k-2)!} = \lambda^2 e^{-\lambda} e^{\lambda} = \lambda^2$$

$$Var(X) = (\lambda^2 + \lambda) - \lambda^2 = \lambda$$

 # For Poisson's dist. both mean and variance are λ.

3.2 Continuous Distributions

 # Total area = 1 ofc

3.2.1 Uniform

pdf $\rightarrow f(x) = \frac{1}{b-a}$ for $a \leq x \leq b$ ($a \rightarrow$ lower limit, $b \rightarrow$ upper limit)

- **mean** $\rightarrow E[X] = \frac{a+b}{2}$

$$\int_a^b x \cdot \frac{1}{b-a}\, dx = \frac{1}{2(b-a)}[x^2]_a^b = \frac{1}{2(b-a)}(b^2 - a^2) = \frac{a+b}{2}$$

- **variance** $\rightarrow Var(X) = \frac{(b-a)^2}{12}$

$$Var(X) = E[X^2] - (E[X])^2$$

$$E[X^2] = \int_a^b x^2 \cdot \frac{1}{b-a}\, dx = \frac{1}{3(b-a)}[x^3]_a^b = \frac{b^3 - a^3}{3(b-a)} = \frac{b^2 + a^2 + ab}{3}$$

$$\begin{aligned} Var(X) &= \frac{b^2 + a^2 + ab}{3} - \left(\frac{a+b}{2}\right)^2 \\ &= \frac{b^2 + a^2 + ab}{3} - \frac{a^2 + b^2 + 2ab}{4} \\ &= \frac{4a^2 + 4b^2 + 4ab - 3a^2 - 3b^2 - 6ab}{12} \\ &= \frac{a^2 + b^2 - 2ab}{12} = \frac{(b-a)^2}{12} \end{aligned}$$

3.2.2 Normal (or Gaussian)

Here mean (μ) and variance (σ^2) are the parameters for defining the distribution. (Bell curve)

pdf $\rightarrow f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$

- **mean** $\rightarrow E[X] = \mu$

$$E[X] = \int_{-\infty}^{\infty} x \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}\, dx$$

$$\text{put } z = \frac{x-\mu}{\sigma} \text{ i.e. } x = z\sigma + \mu,\ dx = \sigma dz$$

$$= \int_{-\infty}^{\infty} \frac{z\sigma + \mu}{\sqrt{2\pi\sigma^2}} e^{-\frac{z^2}{2}} \sigma\, dz$$

$$= \frac{\sigma}{\sqrt{2\pi}} \underbrace{\int_{-\infty}^{\infty} z e^{-\frac{z^2}{2}}\, dz}_{\text{odd fn} \rightarrow 0} + \mu \quad \underbrace{\int_{-\infty}^{\infty} \frac{1}{\sqrt{2\pi}} e^{-\frac{z^2}{2}}\, dz}_{\text{Standard Normal dist, AUC=1}}$$

$$= 0 + \mu(1) = \mu$$

- **variance** $\rightarrow Var(X) = \sigma^2$

$$Var(X) = E[(X-\mu)^2] = \int_{-\infty}^{\infty} (x-\mu)^2 \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}\, dx$$

$$\text{put } z = \frac{x-\mu}{\sigma} \text{ i.e. } x = z\sigma + \mu,\ dx = \sigma dz$$

$$= \int_{-\infty}^{\infty} \frac{z^2\sigma^2}{\sqrt{2\pi\sigma^2}} e^{-\frac{z^2}{2}} \sigma\, dz$$

$$= \sigma^2 \int_{-\infty}^{\infty} \frac{1}{\sqrt{2\pi}} z^2 e^{-\frac{z^2}{2}}\, dz$$

Use by parts:

$$= \sigma^2 \frac{1}{\sqrt{2\pi}} \left(\left[-z e^{-z^2/2} \right]_{-\infty}^{\infty} + \int_{-\infty}^{\infty} e^{-\frac{z^2}{2}}\, dz \right)$$

$$= \sigma^2 \left(0 + \underbrace{\int_{-\infty}^{\infty} \frac{1}{\sqrt{2\pi}} e^{-\frac{z^2}{2}}\, dz}_{\text{Standard Normal Dist.}} \right)$$

$$= \sigma^2$$

4 Independence, Covariance, and Conditional Probability

4.1 Independence and Covariance

- A and B are independent if $P(A \cap B) = P(A)P(B)$.

- Random variables X, Y are independent if their joint distribution factors:

$$f(x, y) = f_X(x) \cdot f_Y(y)$$

(Marginal probability dist $X = x$ and ignore Y)

- For independent X & $Y \rightarrow E[XY] = E[X]E[Y]$ and $Var(X + Y) = Var(X) + Var(Y)$.

 # Covariance measures dir^{n} of dependence

$$Cov(X, Y) = E[XY] - E[X]E[Y]$$

The correlation coefficient normalises to $[-1, 1]$:

$$\rho(X, Y) = \frac{Cov(X, Y)}{\sigma_X \sigma_Y}$$

- $\rho = 0 \rightarrow$ uncorrelated but not necessarily independent.
- $\rho = 1 \rightarrow$ perfect +ve linear relationship (converse is true tho).
- $\rho = -1 \rightarrow$ perfect -ve linear relationship.

4.2 Conditional Probability and Bayes' Theorem

$$P(A|B) = \frac{P(A \cap B)}{P(B)}$$

Bayes' Thm:

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}$$

Introduction to Markov Chains

5 Stochastic Process

A stochastic process is simply a collection of random variables, usually indexed by time:
$\{X_t, t \in T\}$
where, t is time (discrete or continuous)
X_t is the state of the process at time t

The set of all possible values that X_t can take is called the **state space**.

5.1 Markov Property (Memorylessness)

A sequence of random variables is a Markov Chain if it possesses the Markov Property.

Mathematically:

$$P(X_{n+1} = j \mid X_n = i, X_{n-1} = i_{n-1}, \ldots, X_0 = i_0) = P(X_{n+1} = j \mid X_n = i)$$

i.e. The probability of moving to state j tomorrow (X_{n+1}), given the entire history of the system from day zero, is exactly the same as the probability of moving to state j given only where we are today ($X_n = i$).

5.2 One-Step Transition Probabilities (P_{ij})

The probability of transitioning from state i to state j in exactly one time step is P_{ij}.

$$P_{ij} = P(X_{n+1} = j \mid X_n = i)$$

These P_{ij}'s should satisfy two rules:

1. Non-negativity: $P_{ij} \geq 0$
2. Row sums to 1: $\sum_{j=0}^{\infty} P_{ij} = 1 \quad \forall i$

5.3 Transition Probability Matrix (P)

All these individual transition probabilities when packaged into a single square matrix (P)

$$P = \begin{bmatrix} P_{00} & P_{01} & P_{02} & \ldots \\ P_{10} & P_{11} & \ldots & \\ \vdots & \vdots & & \\ P_{i0} & P_{i1} & \ldots & \\ \vdots & \vdots & & \end{bmatrix}$$

6 Chapman - Kolmogorov Equations

Core Problem: We know P_{ij} which is probability of going from i to j in exactly 1 step.

But what if we want to know the probability of going from i to j in exactly n steps? We denote this as $P_{ij}^{(n)}$.

Equation:
To figure out the probability of going from i to j in $n+m$ steps, you have to account for every possible intermediate state k.

Formally:

$$P_{ij}^{(n+m)} = \sum_{k=0}^{\infty} P_{ik}^{(n)} P_{kj}^{(m)}$$

If we let $P^{(n)}$ denote the matrix of n-step transition probabilities $P_{ij}^{(n)}$, then

$$P^{(n+m)} = P^{(n)} \cdot P^{(m)}$$

$\hookrightarrow$ matrix multiplication

7 Examples

From Introduction to Probability Models by Sheldon Ross:

Example 4.10. An urn always contains 2 balls. Ball colors are red and blue. At each stage a ball is randomly chosen and then replaced by a new ball, which with probability 0.8 is the same color, and with probability 0.2 is the opposite color, as the ball it replaces. If initially both balls are red, find the probability that the fifth ball selected is red.

Solution:

states:
state 0 $\rightarrow$ 0 Red 2 Blue
state 1 $\rightarrow$ 1 Red 1 Blue
state 2 $\rightarrow$ 2 Red 0 Blue

Let X_n be the no. of red balls in urn after nth selection and subsequent replacement.

Transition Matrix:

$$P = \begin{bmatrix} 0.8 & 0.2 & 0 \\ 0.1 & 0.8 & 0.1 \\ 0 & 0.2 & 0.8 \end{bmatrix}$$

P(fifth selection is red)

$$= \sum_{i=0}^{2} P(\text{fifth selection is red} \mid X_4 = i) P(X_4 = i \mid X_0 = 2)$$

$$= (0)P_{2,0}^{(4)} + (0.5)P_{2,1}^{(4)} + (1)P_{2,2}^{(4)}$$

$$= 0.7048 \quad \text{(after carrying matrix multiplications)}$$

Example 4.11. Suppose that balls are successively distributed among 8 urns, with each ball being equally likely to be put in any of these urns. What is the probability that there will be exactly 3 nonempty urns after 9 balls have been distributed?

Solution:
$\rightarrow X_n$ be the no of nonempty urns after n balls have been distributed.
states: 0 to 8
$P_{i,i} = i/8$
$P_{i,i+1} = 1 - P_{i,i}$

We want to find $P_{0,3}^{(9)}$.

$$P_{0,3}^{(9)} = P_{0,1}^{(1)} \cdot P_{1,3}^{(8)} = 1 \cdot P_{1,3}^{(8)} = P_{1,3}^{(8)}$$

To build a transition matrix, we have states $1, 2, 3, 4$ (in fact, ≥ 4 nonempty urns).

$$P = \begin{bmatrix} 1/8 & 7/8 & 0 & 0 \\ 0 & 2/8 & 6/8 & 0 \\ 0 & 0 & 3/8 & 5/8 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

we need $P_{1,3}^{(8)}$.

After Matrix multiplications

$$P_{1,3}^{(8)} = 0.00756$$

Example 4.12. In a sequence of independent flips of a fair coin, let N denote the number of flips until there is a run of three consecutive heads. Find
(a) $P(N \leq 8)$ and
(b) $P(N = 8)$.

Solution:
$\rightarrow$ Markov chain, as we are currently on a run of i consecutive heads.
states: $0, 1, 2, 3$ ($3 \rightarrow$ 3 consecutive heads have already occurred)

$$P = \begin{bmatrix} 1/2 & 1/2 & 0 & 0 \\ 1/2 & 0 & 1/2 & 0 \\ 1/2 & 0 & 0 & 1/2 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

We want:
a) $P_{0,3}^{(8)}$
b) $P_{0,2}^{(7)} P_{2,3}^{(1)}$

Calculate P^7 and P^8 to get the answer.

8 State Classification

- State j is said to be accessible from state i if $P_{ij}^n > 0$ for some $n \geq 0$.

Proposition: State j is accessible from state i if and only if, starting in i, it is possible that the process will ever enter state j.

Proof: If j is not accessible from i, then

$$\begin{aligned} P\{\text{ever be in } j \mid \text{start in } i\} &= P\left\{\bigcup_{n=0}^{\infty}\{X_n = j\}\middle| X_0 = i\right\} \\ &\leq \sum_{n=0}^{\infty} P\{X_n = j \mid X_0 = i\} \\ &= \sum_{n=0}^{\infty} P_{ij}^n \\ &= 0 \end{aligned}$$

Two states i and j that are accessible to each other are said to communicate, denoted by $i \leftrightarrow j$.

The relation of communication follows three rules:

1. A state always communicates with itself. (Type of reflexive property)
2. It works both ways. If state i communicates with state j, then state j automatically communicates with state i. (Type of symmetric property)
3. It creates a chain reaction. If state i can reach state j, and state j can reach state k, then state i can definitely reach state k. The math equation in the image simply proves this: if it takes n steps to get to j and m steps to get to k, it will take $n+m$ steps to get from i to k. (Type of transitive property)

Classes: States that all communicate with one another are grouped into a **class**. Two classes are either exactly the same or completely separate (disjoint).

Irreducible: If every single state can reach every other state, the Markov chain is called **irreducible**.

Absorbing State: A state from which no other state is accessible. State i is absorbing if $P_{ii} = 1$.

Let f_i denote the probability that, starting in state i, the process will *ever* reenter state i.

 # Recurrent State: A state i is recurrent if $f_i = 1$. If a process starts in a recurrent state, it will reenter that state with 100% probability. After each reentry, it is like the process again starts over so if a state is recurrent then it will reenter infinitely often.

 # Transient State: A state i is transient if $f_i < 1$. Each time the process enters a transient state, there is a positive probability $(1 - f_i)$ that it will *never* enter it again. The probability of being in state i for exactly n time periods is $f_i^{n-1}(1 - f_i)$ for $n \geq 1$.

- The total number of time periods spent in a transient state follows a geometric distribution with a finite mean of $1/(1 - f_i)$.

Corollary:
If state i is recurrent, and state i communicates with state j, then state j is also recurrent.

Proof:
To prove this, we first note that since state i communicates with state j, there exist integers k and m such that $P_{ij}^k > 0$ and $P_{ji}^m > 0$.

Now, for any integer n:

$$P_{jj}^{m+n+k} \geq P_{ji}^m P_{ii}^n P_{ij}^k$$

This inequality holds because the left side represents the total probability of going from state j to state j in $m + n + k$ steps. The right side represents the probability of going from j to j in $m + n + k$ steps via a specific, restricted path:

1. Going from j to i in exactly m steps.
2. Then going from i to i in an additional n steps.
3. Then going from i to j in an additional k steps.

From the preceding inequality, we obtain, by summing over n, that:

$$\sum_{n=1}^{\infty} P_{jj}^{m+n+k} \geq P_{ji}^m P_{ij}^k \sum_{n=1}^{\infty} P_{ii}^n = \infty$$

This evaluates to infinity because $P_{ji}^m P_{ij}^k > 0$ (as established by the definition of communication), and the summation $\sum_{n=1}^{\infty} P_{ii}^n$ is infinite because state i is known to be recurrent.

Example 4.18. Consider the Markov chain having states 0, 1, 2, 3, 4 and

$$P = \begin{bmatrix} 1/2 & 1/2 & 0 & 0 & 0 \\ 1/2 & 1/2 & 0 & 0 & 0 \\ 0 & 0 & 1/2 & 1/2 & 0 \\ 0 & 0 & 1/2 & 1/2 & 0 \\ 1/4 & 1/4 & 0 & 0 & 1/2 \end{bmatrix}$$

Determine the recurrent state.

Solution: This chain consists of the three classes $\{0, 1\}$, $\{2, 3\}$, and $\{4\}$. The first two classes are recurrent and the third transient.

9 Analysing States in a Random Walk

Consider a Markov chain whose state space consists of the integers $i = 0, \pm 1, \pm 2, \ldots$, and has transition probabilities given by $P_{i,i+1} = p = 1 - P_{i,i-1}$, $i = 0, \pm 1, \pm 2, \ldots$ where $0 < p < 1$.

Since all states clearly communicate, the states are either all transient or all recurrent. So we have to determine whether the summation $\sum_{n=1}^{\infty} P_{00}^n$ is finite or infinite.

It is impossible to reenter the original state exactly in an odd number of moves. It is only possible to reenter the original state in an even number of moves if half the times we go right and half the times left.

$$\begin{aligned} P_{00}^{2n} &= \binom{2n}{n} p^n (1-p)^n \\ &= \frac{(2n)!}{n!n!} (p(1-p))^n, \quad n = 1, 2, 3, \ldots \end{aligned}$$

By using an approximation, due to Stirling, which asserts that:

$$n! \sim n^{n+1/2} e^{-n} \sqrt{2\pi} \tag{4.3}$$

where we say that $a_n \sim b_n$ when $\lim_{n\to\infty} a_n/b_n = 1$, we obtain:

$$\binom{2n}{n} \sim \frac{(2n)^{2n+1/2} e^{-2n} \sqrt{2\pi}}{n^{2n+1} e^{-2n} (2\pi)} = \frac{2^{2n}}{\sqrt{n\pi}}$$

Hence,

$$P_{00}^{2n} \sim \frac{(4p(1-p))^n}{\sqrt{\pi n}}$$

Now it is easy to verify, for positive a_n, b_n, that if $a_n \sim b_n$, then $\sum_n a_n < \infty$ if and only if $\sum_n b_n < \infty$. Hence, $\sum_{n=1}^{\infty} P_{00}^{2n}$ will converge if and only if

$$\sum_{n=1}^{\infty} \frac{(4p(1-p))^n}{\sqrt{\pi n}}$$

does. However, $4p(1-p) \leq 1$ with equality holding if and only if $p = 1/2$. Hence, $\sum_{n=1}^{\infty} P_{00}^{2n} = \infty$ if and only if $p = 1/2$. Thus, the chain is recurrent when $p = 1/2$ and transient if $p \neq 1/2$.

Therefore for $p = 1/2$ all the states are recurrent, elsewhere transient.

Also if we take a 2-dimensional walk i.e. 4 states (right, left, up, down), each having a probability of 1/4, then here too all the states are recurrent.

Interestingly, whereas the symmetric random walks in one and two dimensions are both recurrent, all higher-dimensional symmetric random walks turn out to be transient.

For a 1-dimensional walk we can prove this in another way too:

Let $\beta = P(\text{ever return to 0})$

$$\beta = P(\text{ever return to 0} \mid X_1 = 1)p + P(\text{ever return to 0} \mid X_1 = -1)(1-p)$$

Now, let α be the probability that the Markov chain currently in state i will ever enter state $i-1$, for any i.

$$\alpha = P(\text{ever return} \mid X_1 = 1, X_2 = 0)(1-p) + P(\text{ever return} \mid X_1 = 1, X_2 = 2)p$$

$$= 1 - p + P(\text{ever return} \mid X_1 = 1, X_2 = 2)p$$

$$= 1 - p + p\alpha^2$$

The two roots of this equation are $\alpha = 1$ and $\alpha = (1-p)/p$.

In case of a Symmetric Random Walk, $p = 1/2$, we get $\alpha = 1$, proving that it is recurrent.

Suppose now that $p > 1/2$, therefore:

$$\beta = \alpha p + 1 - p$$

Because the random walk is transient in this case we know that $\beta < 1$, showing that $\alpha \neq 1$.
Therefore, $\alpha = (1-p)/p$,
so, $\beta = 2(1-p),$ for $p > 1/2$.

Similarly, when $p < 1/2$ we can show that $\beta = 2p$.

Thus in general,

$$P\{\text{ever return to 0}\} = 2\min(p, 1-p)$$

10 Long-Run Proportions and Limiting Probabilities

Proposition: If i is recurrent and i communicates with j, then $f_{ij} = 1$.

In short: If state i loops infinitely, and state j can reach state i and come back, then those infinite loops at i drag j into looping infinitely too.

Let $N_j = \min\{n > 0 : X_n = j\}$.
If state j is recurrent, let m_j denote the expected number of transitions that it takes the Markov chain when starting in state j to return to that state.

$$m_j = E[N_j \mid X_0 = j]$$

The recurrent state j is **positive recurrent** if $m_j < \infty$ and it is **null recurrent** if $m_j = \infty$.

Proposition: If the Markov chain is irreducible and recurrent, then for any initial state:

$$\pi_j = \frac{1}{m_j}$$

Proof:
Suppose the process starts in state i.

- Let T_1 = the number of steps to reach state j for the *first* time.
- Let $T_2, T_3, \ldots, T_n$ = the number of steps between subsequent visits to state j.

The n-th visit to state j happens at time $T_1 + T_2 + \cdots + T_n$.

Therefore, the proportion of time spent in state j over the long run is n visits divided by the total time it took to get there:

$$\pi_j = \lim_{n\to\infty} \frac{n}{\sum_{i=1}^{n} T_i}$$

$$\pi_j = \lim_{n\to\infty} \frac{1}{\frac{1}{n}\sum_{i=1}^{n} T_i}$$

$$\pi_j = \lim_{n\to\infty} \frac{1}{\frac{T_1}{n} + \frac{T_2+\cdots+T_n}{n}}$$

As the number of visits (n) approaches infinity:

- $\lim_{n\to\infty} \frac{T_1}{n} = 0$ (Because the time to first reach j is finite, dividing it by infinity yields zero).
- $\lim_{n\to\infty} \frac{T_2+\cdots+T_n}{n} = m_j$ (By the Strong Law of Large Numbers, the average of these repeated, identical return times converges exactly to their expected mean, m_j).

Therefore,

$$\pi_j = \frac{1}{0 + m_j} = \frac{1}{m_j}$$

Imagine a Markov chain running for an infinitely long time. π_j is the exact fraction (or percentage) of that total time the system spends sitting in state j.

If you let the Markov chain run for a very long time, π_j is the probability that you will find the system in state j at some random point far in the future, regardless of where it originally started.

Proposition: If i is positive recurrent and $i \leftrightarrow j$ then j is positive recurrent.

Ergodic Theorem: Consider an irreducible Markov chain. If the chain is positive recurrent, then the long-run proportions are the unique solution of the equations:

$$\pi_j = \sum_i \pi_i P_{ij}, \quad j \geq 0, \quad \sum_j \pi_j = 1$$

Moreover, if there is no solution to the preceding linear equations, then the Markov chain is either transient or null recurrent and all $\pi_j = 0$.

Convergence: As $n \to \infty$, the transition probabilities P_{ij}^n of certain Markov chains converge to a specific value that is independent of the initial state i.

Periodic Chains: A Markov chain is considered periodic if it can only return to a state in a multiple of $d > 1$ steps. Periodic chains do not have limiting probabilities, although they can still have long-run proportions (e.g., a two-state chain that continually alternates between states 0 and 1).

Aperiodic Chains: An irreducible chain that is not periodic is called aperiodic. For these chains, limiting probabilities always exist, do not depend on the initial state, and are equal to the long-run proportions (π_j).

Equivalence to Long-Run Proportions: Because the set $\{\alpha_j, j \geq 0\}$ satisfies the exact same equations for which the long-run proportions $\{\pi_j, j \geq 0\}$ are the unique solution, it proves that $\alpha_j = \pi_j$.

 # Ergodic Chains: A Markov chain that is irreducible, positive recurrent, and aperiodic is called **ergodic**.

Markov Chains (Python Implementation)

11 Markov Chain Object-Oriented Implementation

Below is the Python implementation of a Markov Chain, consolidating core functionalities of what learnt till now such as n-step transitions, calculating stationary distributions, identifying absorbing states, and determining communicating classes using the Floyd-Warshall algorithm.

```python
import numpy as np

class MarkovChain:
    def __init__(self, states, transition_matrix):
        # Initializing our Markov Chain.

        self.states = states
        self.transition_matrix = np.array(transition_matrix)

        # Validate the matrix by its 2 basic rules
        assert self.transition_matrix.shape[0] == self.transition_matrix.shape[1], "Matrix must be a square."
        for row in self.transition_matrix:
            assert np.isclose(sum(row), 1.0), f"Row probabilities must sum to 1. Found: {sum(row)}"

    def get_n_step_transition(self, n):
        # To Calculate the n-step transition matrix using matrix exponentiation.
        return np.linalg.matrix_power(self.transition_matrix, n)

    def calculate_stationary_distribution(self):
        # To Calculate the stationary distribution (pi) where pi*P = pi.
        # This is equivalent to finding the left eigenvector associated with eigenvalue 1.

        # First Transpose the matrix to find left eigenvectors using standard right eigenvector functions
        eigenvalues, eigenvectors = np.linalg.eig(self.transition_matrix.T)
        # Then find the index of the eigenvalue that is closest to 1
        eigen_1_index = np.argmin(np.abs(eigenvalues - 1.0))
        # Then extract the corresponding eigenvector
        stationary_vector = eigenvectors[:, eigen_1_index].real
        # And finally normalize the vector so probabilities sum to 1
        stationary_distribution = stationary_vector / np.sum(stationary_vector)

        return stationary_distribution

    def display_stationary(self):
        # To print the stationary distribution in a readable format
        pi = self.calculate_stationary_distribution()
```

```
        print("\n>>> Stationary Distribution <<<")
        for state, prob in zip(self.states, pi):
            print(f"State {state}: {prob:.4f} ({prob*100:.2f}%)")

    def get_absorbing_states(self):
        # To get absorbing states
        absorbing = []
        for i, state in enumerate(self.states):
            if np.isclose(self.transition_matrix[i, i], 1.0):
                absorbing.append(state)
        return absorbing

    def simulate_path(self, start_state, num_steps):
        # To simulates a random walk through the Markov Chain.

        # The starting state should be a valid state ofc
        if start_state not in self.states:
            raise ValueError(f"State '{start_state}' not found in state space.")

        current_state_idx = self.states.index(start_state)
        path = [start_state]

        for i in range(num_steps):
            next_state_idx = np.random.choice(
                len(self.states),
                p=self.transition_matrix[current_state_idx]
            )
            path.append(self.states[next_state_idx])
            current_state_idx = next_state_idx

        return path

    def get_communicating_classes(self):
        # To identify communicating classes (Strongly Connected Components) using reachability to help classify states as transient or recurrent.

        n = len(self.states)
        # First we will need a Boolean reachability matrix (which answers, can i reach j directly?)
        reachability = np.zeros((n, n), dtype=bool)
        np.fill_diagonal(reachability, True)
        reachability = reachability | (self.transition_matrix > 0)

        # Using Floyd-Warshall algorithm
        for k in range(n):
            for i in range(n):
                for j in range(n):
                    reachability[i, j] = reachability[i, j] or (reachability[i, k] and reachability[k, j])

        # Group states into communicating classes
        visited = set()
        classes = []
        for i in range(n):
            if i not in visited:
```

```
                # A class includes all states 'j' where 'i' can reach 'j' AND 'j' can reach 'i'
                current_class_indices = [j for j in range(n) if reachability[i, j] and reachability[j, i]]
                classes.append([self.states[idx] for idx in current_class_indices])
                visited.update(current_class_indices)

        return classes


if __name__ == "__main__":
    # EXAMPLE 1: Weather Model

    # Defining the states
    weather_states = ['Sunny', 'Cloudy', 'Rainy']

    # Defining the Transition Matrix (P)
    # Row 1 (Sunny): 60% chance tomorrow is Sunny, 30% Cloudy, 10% Rainy
    # Row 2 (Cloudy): 40% Sunny, 40% Cloudy, 20% Rainy
    # Row 3 (Rainy): 20% Sunny, 50% Cloudy, 30% Rainy
    P = [[0.6, 0.3, 0.1],
         [0.4, 0.4, 0.2],
         [0.2, 0.5, 0.3]]

    # Initializing the chain
    mc = MarkovChain(weather_states, P)

    # Q. What happens after 5 days?
    print("\n>>> 5-Step Transition Matrix (P^5) <<<<")
    print(np.round(mc.get_n_step_transition(5), 4))

    # Q. Calculate the Stationary Distribution
    mc.display_stationary()

    # Q. Simulate 10 days of weather starting from a Sunny day
    print("\n>>> 10-Day Weather Simulation <<<")
    weather_path = mc.simulate_path('Sunny', 10)
    print(" -> ".join(weather_path))


    # EXAMPLE 2: The Gambler's Ruin (Absorbing States & Classes)
    print("Example 2: Gambler's Ruin ($0 to $3)")

    gambler_states = ['$0', '$1', '$2', '$3']
    # p = 0.5 (fair coin flip to win or lose $1)
    gambler_P = [
        [1.0, 0.0, 0.0, 0.0],
        [0.5, 0.0, 0.5, 0.0],
        [0.0, 0.0, 0.0, 1.0],
        [0.5, 0.0, 0.5, 0.0]
    ]

    gambler_mc = MarkovChain(gambler_states, gambler_P)

    print("\n>>> Absorbing States <<<")
    print(gambler_mc.get_absorbing_states())
```

```
143
144     print("\n>>> Communicating Classes <<<")
145     classes = gambler_mc.get_communicating_classes()
146     for i, c in enumerate(classes):
147         # A class with only 1 state that is absorbing is recurrent.
148         # A class that can reach an absorbing state but isn't absorbing
    itself is transient.
149         print(f"Class {i+1}: {c}")
150
151     print("\n>>> Gambler Simulation (Starting at $1) <<<")
152     gambler_path = gambler_mc.simulate_path('$1', 7)
153     print(" -> ".join(gambler_path))
```

Exponential Distribution and the Poisson Process

12 The Exponential Distribution

A continuous random variable X is said to have an exponential distribution with parameter λ, where $\lambda > 0$, if its probability density function (PDF) is given by:

$$f(x) = \begin{cases} \lambda e^{-\lambda x}, & x \geq 0 \\ 0, & x < 0 \end{cases}$$

Equivalently, its cumulative distribution function (CDF) is given by:

$$F(x) = \int_{-\infty}^{x} f(y)dy = \begin{cases} 1 - e^{-\lambda x}, & x \geq 0 \\ 0, & x < 0 \end{cases}$$

12.1 Mean and Variance

Mean ($E[X]$)
The mean is calculated using integration by parts where $u = x$, $dv = \lambda e^{-\lambda x}dx$:

$$\begin{aligned} E[X] &= \int_0^\infty x\lambda e^{-\lambda x}dx \\ &= \left[-xe^{-\lambda x}\right]_0^\infty + \int_0^\infty e^{-\lambda x}dx \\ &= 0 + \left[-\frac{1}{\lambda}e^{-\lambda x}\right]_0^\infty \\ &= \frac{1}{\lambda} \end{aligned}$$

Variance ($Var(X)$)
First, compute $E[X^2]$ using integration by parts ($u = x^2$, $dv = \lambda e^{-\lambda x}dx$):

$$\begin{aligned} E[X^2] &= \int_0^\infty x^2\lambda e^{-\lambda x}dx \\ &= \left[-x^2e^{-\lambda x}\right]_0^\infty + 2\int_0^\infty xe^{-\lambda x}dx \\ &= 2 \cdot \frac{1}{\lambda^2} = \frac{2}{\lambda^2} \end{aligned}$$

Now, compute the variance:

$$\begin{aligned} Var(X) &= E[X^2] - (E[X])^2 \\ &= \frac{2}{\lambda^2} - \left(\frac{1}{\lambda}\right)^2 \\ &= \frac{1}{\lambda^2} \end{aligned}$$

12.2 The Memoryless Property

A random variable X is said to be without memory, or *memoryless*, if:

$$P(X > s + t \mid X > t) = P(X > s) \quad \text{for all } s, t \geq 0$$

This is equivalent to stating:

$$P(X > s + t) = P(X > s)P(X > t)$$

Example:
Suppose the amount of time spent in a bank is exponentially distributed with a mean of 10 minutes ($\lambda = \frac{1}{10}$).

Probability of spending more than 15 minutes:

$$P(X > 15) = e^{-15\lambda} = e^{-3/2} \approx 0.223$$

Probability of spending more than 15 minutes GIVEN she is still there after 10 minutes:
Because of the memoryless property, this is simply the probability of spending at least 5 *more* minutes:

$$P(X > 5) = e^{-5\lambda} = e^{-1/2} \approx 0.607$$

12.3 Properties of Multiple Exponential Variables

Proposition: If $X_1, \ldots, X_n$ are independent exponential random variables with a common rate λ, then their sum $\sum_{i=1}^{n} X_i$ is a gamma (n, λ) random variable with density function:

$$f(t) = \lambda e^{-\lambda t} \frac{(\lambda t)^{n-1}}{(n-1)!}, \quad t > 0$$

Probability of one variable being smaller than another:
If X_1 and X_2 are independent with rates λ_1 and λ_2:

$$\begin{aligned} P(X_1 < X_2) &= \int_0^\infty P(X_1 < X_2 \mid X_1 = x)\lambda_1 e^{-\lambda_1 x} dx \\ &= \int_0^\infty e^{-\lambda_2 x} \lambda_1 e^{-\lambda_1 x} dx \\ &= \frac{\lambda_1}{\lambda_1 + \lambda_2} \end{aligned}$$

13 The Poisson Process

Counting Processes:
A stochastic process $\{N(t), t \geq 0\}$ is a counting process if $N(t)$ represents the total number of "events" that occur by time t. It must satisfy:

1. $N(t) \geq 0$
2. $N(t)$ is integer-valued.
3. If $s < t$, then $N(s) \leq N(t)$.
4. For $s < t$, $N(t) - N(s)$ equals the number of events that occur in the interval $(s, t]$.

Independent Increments: A counting process possesses independent increments if the numbers of events that occur in disjoint time intervals are independent.

Little-o Notation (Definition):
The function $f(\cdot)$ is said to be $o(h)$ if:

$$\lim_{h \to 0} \frac{f(h)}{h} = 0$$

Definition (The Poisson Process):
The counting process $\{N(t), t \geq 0\}$ is a Poisson process with rate $\lambda > 0$ if the following axioms hold:

1. $N(0) = 0$
2. The process has independent increments.
3. $P(N(t+h) - N(t) = 1) = \lambda h + o(h)$
4. $P(N(t+h) - N(t) \geq 2) = o(h)$

13.1 Interarrival Times and Sequence Distributions

Let T_1 denote the time of the first event, and T_n denote the time between the $(n-1)$st and nth event. The sequence $\{T_n, n = 1, 2, \dots\}$ is the sequence of interarrival times.

Lemma:
$T_1, T_2, \dots$ are independent and identically distributed exponential random variables with rate λ.

Proof for T_1:

$$P(T_1 > t) = P(N(t) = 0) = e^{-\lambda t}$$

Letting $S_n = \sum_{i=1}^{n} T_i$ be the time of the nth event, it follows from Proposition 5.1 that S_n is a gamma (n, λ) random variable:

$$f_{S_n}(s) = \lambda e^{-\lambda s} \frac{(\lambda s)^{n-1}}{(n-1)!}, \quad s > 0$$

Theorem :
If $\{N(t), t \geq 0\}$ is a Poisson process with rate λ, then $N(t)$ is a Poisson random variable with rate λt. That is:

$$P(N(t) = n) = e^{-\lambda t} \frac{(\lambda t)^n}{n!}, \quad n \geq 0$$

Introduction to MDPs & RL Core Concepts

14 Introduction to Reinforcement Learning

- **Core Concept:** RL is based on an agent learning by trial and error. Unlike supervised learning, the agent is not explicitly told what the correct action is; it must discover which actions yield the most reward by interacting with the world.
- **Reward Hypothesis:** All goals can be described by the maximization of expected total reward. If we want an agent to achieve a specific goal, we must provide rewards to it in such a way that maximizing those rewards corresponds perfectly with achieving the goal.
- **Delayed Rewards:** Rewards may be delayed, meaning an action taken now can result in positive or negative rewards much later in the future (after multiple steps in a trajectory). This introduces the challenge of *credit assignment*, determining which past actions were responsible for a future outcome.

14.1 The Agent-Environment Interaction

Environment: The world that the agent lives in, interacts with, and learns from. The environment defines the rules of the universe, including state transitions and reward logic.

The Loop: The interaction unfolds in a sequence of discrete time steps, $t = 0, 1, 2, 3, \ldots$ At each step t:

- **Agent:** Executes an action A_t, receives an observation O_t, and receives a scalar reward R_t.
- **Environment:** Receives the action A_t, updates its internal state, emits the next observation O_{t+1}, and emits a scalar reward R_{t+1}.

In summary: The agent sees a state $\rightarrow$ takes an action $\rightarrow$ receives a reward signal and the next state from the environment.

14.2 History, State, and Observability

History and State

- **History (H_t):** The complete sequence of observations, actions, and rewards up to time t:
$$H_t = O_1, R_1, A_1, \ldots, A_{t-1}, O_t, R_t$$
What happens next depends entirely on the history.

- **State (S_t):** The concise information used to determine what happens next. Formally, state is a function of the history: $S_t = f(H_t)$. It is a complete description of the world, whereas an **Observation (O_t)** is only a partial description.
- **Trajectories (τ):** A specific sequence of states, actions, and rewards over an episode.

Types of State

- **Environment State (S_t^e):** The environment's private representation. It contains whatever data the environment uses to pick the next observation/reward. It is not usually visible to the agent and often contains irrelevant or redundant information.
- **Agent State (S_t^a):** The agent's internal representation. It contains whatever information the agent uses to pick the next action. This is the information used and updated by RL algorithms.
- **Information State (Markov State):** Contains all useful information from the history. A state S_t is Markov if and only if it follows the Markov Property:

$$P[S_{t+1} \mid S_t] = P[S_{t+1} \mid S_1, \ldots, S_t]$$

"The future is independent of the past given the present." Once the Markov state is known, the history may be thrown away.

Observability

- **Fully Observable Environments:** The agent directly observes the complete environment state ($O_t = S_t^a = S_t^e$). This is formally modeled as a **Markov Decision Process (MDP)**.
- **Partially Observable Environments:** The agent indirectly observes the environment (e.g., a robot with limited camera vision). Here, $S_t^a \neq S_t^e$. Formally, this is a **Partially Observable Markov Decision Process (POMDP)**. The agent must construct its own state representation, perhaps using beliefs or recurrent neural networks.

15 Major Components of an RL Agent

An RL agent typically includes one or more of the following key elements, operating within an **Action Space** ($\mathcal{A}$), which is the set of all valid actions available to the agent.

1. Policy (π)

- The agent's behavior function, serving as a map from the perceived states of the environment to the actions to be taken. It is the core of the RL agent.
- **Deterministic Policy:** $a = \pi(s)$. Maps a state exactly to one specific action.

- **Stochastic Policy:** $\pi(a \mid s) = P[A_t = a \mid S_t = s]$. Outputs a probability distribution over actions, which is highly useful for exploration.

2. Reward (R) and Return (G)

- **Reward Signal:** Defines the immediate, short-term goal of the RL problem.
- **Return (G_t):** The total expected cumulative reward from time step t.
- **Discount Factor ($\gamma \in [0, 1]$):** A mathematical factor used to ensure the sum of future rewards converges in infinite-horizon problems, and to represent uncertainty about the future. The discounted return is defined as:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

  If $\gamma = 0$, the agent is myopic (only cares about immediate reward). As γ approaches 1, the agent becomes farsighted.

3. Value Functions

- Functions that predict the expected return to evaluate the goodness or badness of situations, helping the agent select between actions.
- **State-Value Function ($V_\pi(s)$):** Measures how good it is to be in a specific state s under policy π.

$$V_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]$$

- **Action-Value Function ($Q_\pi(s, a)$):** Measures how good it is to take a specific action a in state s under policy π.

$$Q_\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a]$$

- **Optimal Value Functions (V^*, Q^*):** Represent the maximum possible expected return an agent can achieve across all possible policies.

4. Model (Optional)

- The agent's internal representation used to predict what the environment will do next. Agents that use models are called **Model-Based** (using planning), while those that don't are **Model-Free** (learning directly from experience).
- **Transitions (Dynamics):** Predicts the next state: $\mathcal{P}^a_{ss'} = P[S_{t+1} = s' \mid S_t = s, A_t = a]$.
- **Rewards:** Predicts the next immediate reward: $\mathcal{R}^a_s = \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a]$.

16 Exploration vs. Exploitation

This is the fundamental trade-off the agent faces while learning:

- **Exploration:** Trying out new, untested actions to gather more information about the environment. This helps discover better long-term strategies.
- **Exploitation:** Using known information to choose the action that is currently expected to yield the highest reward to maximize short-term gains.
- **Common Strategies:** A standard approach to balance this is the ϵ**-greedy** strategy, where the agent exploits the best known action with probability $1 - \epsilon$, and explores a random action with probability ϵ.

DP Algorithms & Model-Free RL

17 Solving Known MDPs: Dynamic Programming

1) Policy Evaluation
Policy evaluation is the process of calculating the state-value function ($V^{\pi}(s)$). It tells us the goodness or badness of a certain policy (π). For this, we use the Bellman Expectation Equation for the state-value function:

$$V_{k+1}(s) = \sum_{a} \pi(a|s) \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma V_k(s') \right]$$

Pseudocode:

- **Input:**
  - The policy π which we want to evaluate.
  - An estimation threshold $\theta > 0$ to determine the accuracy of our estimation.
  - Transition probabilities / Transition Matrix.
  - Discount factor $\gamma \in [0, 1]$.
- **Process:**
  - Set the value of every state arbitrarily.
  - Set the value of the terminal state to 0.
  - Set Δ to 0.
  - **Loop** until $\Delta < \theta$:
    - For each state in the state space:
    - Store the current value of the state in a variable.
    - Update the value of the state using the Bellman Expectation Equation.
    - Check how much the state's value changed.
    - Set $\Delta \leftarrow \max(\text{current } \Delta, \text{absolute difference between the old and new state value})$.
- **Output:** The final value of each state, which is now a close approximation of the value function $V^{\pi}(s)$ for the given policy.

2) Policy Iteration
Policy Iteration is an algorithm used to discover the optimal policy.

Pseudocode:

- **Input:**

  - An estimation threshold $\theta > 0$ to determine the accuracy of our estimation.
  - Transition probabilities / Transition Matrix.
  - Discount factor $\gamma \in [0, 1]$.

- **Process:**
  - Set the value of every state arbitrarily.
  - Set Δ to 0.
  - Start with an arbitrary policy and perform **Policy Evaluation**.
  - Then perform **Policy Improvement**, for each state:
    * Store the current action and calculate the expected return for all possible actions.
    * Select the action with the highest expected return.
    * Update the policy to choose this action.
    * If the selected action differs from the previous action, mark the policy as unstable.
  - Check whether the policy changed: If no action changed in any state, stop; otherwise, repeat Policy Evaluation with the improved policy.

- **Output:** Optimal policy π^* and the optimal value function $V^*(s)$ corresponding to that policy.

3) Value Iteration

Value Iteration is an algorithm which helps in finding the optimal strategy faster. For this, we use the Bellman Optimality Equation of the state-value function:

$$V_{k+1}(s) = \max_a \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma V_k(s') \right]$$

Pseudocode:

- **Input:**
  - An estimation threshold $\theta > 0$ to determine the accuracy of our estimation.
  - Transition probabilities / Transition Matrix.
  - Discount factor $\gamma \in [0, 1]$.

- **Process:**
  - Set the value of every state arbitrarily.
  - Set the value of the terminal state to 0.
  - **Loop** until the maximum change in state values (Δ) becomes smaller than θ:
    * Set Δ to 0.
    * Repeatedly update the value of each state using the Bellman Optimality Equation.

  - Extract the optimal policy by choosing, in each state, the action that yields the highest expected return according to the converged value function.

- **Output:** Optimal policy π^* and the optimal value function $V^*(s)$ corresponding to that policy.

18 Introduction to Model-Free RL: MC vs TD

We will take an example of a small toy MDP for this comparison to highlight how learning from experience differs from Dynamic Programming.

Toy MDP Setup
Imagine a 2×2 grid:

- Top Left (TL) $\rightarrow$ Top Right (TR)
- Bottom Left (BL) $\rightarrow$ Bottom Right (BR)

BR is our Goal.

- **Actions:** Up, Down, Left, Right.
- **Rewards:** Every step taken costs -1 & stepping into goal (BR) gives $+10$.
- **Discount factor (γ):** 1 (i.e., no discounting).
- **Learning Rate (α):** 0.5.

Initially, $V(TL) = 0,\ V(TR) = 0,\ V(BL) = 0,\ V(BR) = 0.$

Let's simulate one specific episode starting at BL:

$$\text{BL} \xrightarrow{\text{UP}} \text{TL}\ (R = -1) \xrightarrow{\text{RIGHT}} \text{TR}\ (R = -1) \xrightarrow{\text{DOWN}} \text{BR}\ (R = +10)$$

Now compare between Monte Carlo and Temporal Difference algorithms:

1) Monte Carlo (MC) Execution
MC must wait until it hits a terminal state as it calculates cumulative return (G_t) from each state.

Work Backwards:

- From TR $\rightarrow$ only one step and total reward is 10. $\therefore G_{TR} = 10.$
- From TL $\rightarrow G_{TL} = (-1) + 10 = 9.$
- From BL $\rightarrow G_{BL} = (-1) + (-1) + 10 = 8.$

Now update the values:

- **Update TR** $\rightarrow V(TR) \leftarrow V(TR) + \alpha[G_{TR} - V(TR)]$
  $V(TR) \leftarrow 0 + 0.5[10 - 0] = 5$
- **Update TL** $\rightarrow V(TL) \leftarrow V(TL) + \alpha[G_{TL} - V(TL)]$
  $V(TL) \leftarrow 0 + 0.5[9 - 0] = 4.5$
- **Update BL** $\rightarrow V(BL) \leftarrow V(BL) + \alpha[G_{BL} - V(BL)]$
  $V(BL) \leftarrow 0 + 0.5[8 - 0] = 4$

We got: $V(TR) = 5,\ V(TL) = 4.5,\ V(BL) = 4.$

2) Temporal Difference (TD) Execution
TD updates the value immediately after every move, bootstrapping off the next state's current estimate.

- **First step (BL $\rightarrow$ TL)** $\rightarrow R + \gamma V(TL) = -1 + 1(0) = -1$
  $V(BL) \leftarrow V(BL) + 0.5[-1 - 0] = -0.5$
- **Second step (TL $\rightarrow$ TR)** $\rightarrow R + \gamma V(TR) = -1 + 1(0) = -1$
  $V(TL) \leftarrow V(TL) + 0.5[-1 - 0] = -0.5$
- **Third step (TR $\rightarrow$ BR)** $\rightarrow R + \gamma V(BR) = 10 + 1(0) = 10$
  $V(TR) \leftarrow V(TR) + 0.5[10 - 0] = 5$

We got: $V(TR) = 5,\ V(TL) = -0.5,\ V(BL) = -0.5.$

Value Comparison

	MC	**TD**
$V(TR)$	5	5
$V(TL)$	4.5	-0.5
$V(BL)$	4	-0.5

Q-Learning in a Stochastic Gridworld

19 Problem Statement

The objective of this implementation is to demonstrate how a model-free Reinforcement Learning agent (using Q-Learning) adapts its policy when navigating an environment with varying degrees of uncertainty.

In classic deterministic pathfinding algorithms (such as A* or Dijkstra's), an agent assumes perfect execution of its actions. However, real-world environments are inherently stochastic. If a robot attempts to move forward, wheel slip or uneven terrain might cause it to veer off course.

We aim to model this uncertainty using a Markov Decision Process (MDP) in a 5×5 Gridworld. The environment contains a highly rewarding goal state and a series of dangerous "pit" states. By manipulating the "slip probability"—the chance that an agent moves perpendicularly to its intended direction—we aim to observe how the optimal policy transitions from a reckless shortest-path strategy to a cautious, risk-averse strategy.

19.1 Proposed Solution

To solve this, we propose building a Tabular Q-Learning agent. The solution comprises two main architectural components:

- **The MDP Environment (`StochasticGridworld`):** A custom grid environment that takes a `slip_probability` parameter. It strictly enforces the rules of the MDP, calculating next states and rewards based on the agent's actions and a random number generator to simulate slipping.
- **The Agent (`QLearningAgent`):** A model-free learning algorithm that interacts with the environment. It uses an ϵ-greedy strategy to explore the grid and updates a $5 \times 5 \times 4$ Q-table using the Bellman Optimality Equation.

By training two separate agents—one in a deterministic environment (slip = 0.0) and one in a stochastic environment (slip = 0.3)—we can directly compare their converged policies.

19.2 Code Implementation & Markov Chain Integration

The Python implementation explicitly models the components of a Markov Decision Process (S, A, P, R, γ).

The Environment (State, Actions, Transitions, Rewards)

- **State Space (S):** Represented by `self.current_state = (x, y)`. The grid is 5×5, creating 25 distinct discrete states.
- **Action Space (A):** `self.actions = [0, 1, 2, 3]` mapping to Up, Right, Down, and Left.
- **Rewards (R):** Handled in the `step()` function. The agent receives $+10$ for reaching the goal $(4, 4)$, -10 for falling into a pit, and a -0.1 step penalty to encourage efficiency.
- **Transition Probabilities (P):** This is the core Markov element of the implementation. In the `step(action)` function, the environment rolls a random variable `roll = np.random.rand()`.
  - If `roll < slip_prob / 2`, the agent slips left of its intended direction.
  - If `roll < slip_prob`, the agent slips right.
  - Otherwise, it moves exactly as intended.

  This logic effectively acts as the transition matrix $P(s' \mid s, a)$ without the agent explicitly knowing the probabilities.

The Q-Learning Agent

The agent initializes a Q-table of zeros: `np.zeros((5, 5, 4))`. It learns purely from the `(state, action, reward, next_state)` tuples returned by the environment.

The core learning mechanism is the Bellman update:

```
td_target = reward + self.gamma * best_next_q
td_error = td_target - self.q_table[x, y, action]
self.q_table[x, y, action] += self.alpha * td_error
```

Because the transitions are stochastic, the agent falls into the pits multiple times during training. The negative reward propagates backward through the Q-table, mathematically lowering the expected value of taking actions near the pits.

19.3 Results and Analysis

After training both agents, we extracted the optimal policy `np.argmax(self.q_table[x, y])` for each state. The results clearly demonstrate the agent's adaptation to stochasticity.

Scenario A: Deterministic Environment (Slip Probability = 0.0)

Output Policy:

```
[ → ] [ → ] [ → ] [ → ] [ G ]
[ → ] [ → ] [ ↑ ] [ ↑ ] [ ↑ ]
[ ↑ ] [ ↑ ] [ ↑ ] [ ↑ ] [ ↑ ]
[ ↑ ] [ X ] [ X ] [ X ] [ ↑ ]
[S↑ ] [ ← ] [ → ] [ → ] [ ↑ ]
```

Analysis: In the deterministic environment, the agent has perfect control. The step penalty (-0.1) drives the agent to find the absolute shortest path to the goal. From the Start state (S), the policy dictates moving Up along the left wall, and then immediately cutting Right. It willingly travels directly adjacent to the deadly pits (X) because the probability of accidentally falling in is 0%. It maximizes reward by minimizing distance.

Scenario B: Stochastic Environment (Slip Probability = 0.3)

Output Policy:

```
[ → ] [ → ] [ → ] [ → ] [ G ]
[ → ] [ → ] [ → ] [ ↑ ] [ ↑ ]
[ ↑ ] [ ↑ ] [ ↑ ] [ ↑ ] [ ↑ ]
[ ← ] [ X ] [ X ] [ X ] [ → ]
[S↑ ] [ ↓ ] [ ↓ ] [ ↓ ] [ ↑ ]
```

Analysis: The introduction of a 30% slip probability fundamentally alters the geometry of the agent's optimal path.

- **Risk Avoidance:** Notice the tiles immediately below the pits. The policy now dictates moving Down (↓) instead of Up or Right. The agent actively pushes itself against the bottom wall. Why? Because if it tries to move Right while adjacent to a pit, a slip could result in an Upward movement, throwing it into the -10 trap.
- **The Wide Detour:** From the Start state, the agent moves Up (↑), but instead of cutting Right near the pits, the arrows facing the pits point Left (←). The agent forces itself against the left wall, traveling all the way to the top-left corner before moving Right across the top row to the Goal.
- **Expected Value:** The agent has mathematically calculated that the cumulative cost of taking extra steps (at -0.1 per step) is far less punishing than the expected value loss of a 15% chance of slipping into a -10 pit.

Conclusion: The experiment successfully demonstrates that Tabular Q-Learning can implicitly learn the stochastic transition dynamics of a Markov environment. Without

being explicitly programmed with the slip probability, the agent discovers and adopts a risk-averse policy purely through experiential learning and the recursive nature of the Bellman equation.

Python Implementation (`iceGrid.py`)

20 Source Code

The following code provides the complete implementation of the Stochastic Gridworld environment and the Tabular Q-Learning agent.

```
import numpy as np

class StochasticGridworld:
    def __init__(self, slip_probability=0.2):
        """
        Initializes the 5x5 Stochastic Gridworld.
        Start: (0, 0)
        Goal: (4, 4) [Reward +10]
        Pits: (1, 1), (2, 1), (3, 1) [Reward -10]
        Step Penalty: -0.1
        """
        self.grid_size = 5
        self.slip_prob = slip_probability
        self.start_state = (0, 0)
        self.goal_state = (4, 4)
        self.pits = [(1, 1), (2, 1), (3, 1)]
        self.current_state = self.start_state

        # Actions: 0: Up, 1: Right, 2: Down, 3: Left
        self.actions = [0, 1, 2, 3]

    def reset(self):
        self.current_state = self.start_state
        return self.current_state

    def step(self, action):
        """
        Takes an action and returns (next_state, reward, is_done).
        Incorporates the slip probability for stochastic transitions.
        """
        if self.current_state == self.goal_state or self.current_state
   in self.pits:
            return self.current_state, 0, True

        # Determine actual movement based on slip probability
        actual_action = action
        roll = np.random.rand()

        if roll < self.slip_prob:
            # Slipped! Move perpendicular
            if roll < self.slip_prob / 2:
                actual_action = (action - 1) % 4 # Slip left
            else:
                actual_action = (action + 1) % 4 # Slip right

        # Calculate new position
        x, y = self.current_state
```

```
        if actual_action == 0: y += 1   # Up
        elif actual_action == 1: x += 1 # Right
        elif actual_action == 2: y -= 1 # Down
        elif actual_action == 3: x -= 1 # Left

        # Keep within bounds
        x = max(0, min(x, self.grid_size - 1))
        y = max(0, min(y, self.grid_size - 1))
        next_state = (x, y)

        # Calculate reward
        reward = -0.1 # Default step penalty
        is_done = False

        if next_state == self.goal_state:
            reward = 10.0
            is_done = True
        elif next_state in self.pits:
            reward = -10.0
            is_done = True

        self.current_state = next_state
        return next_state, reward, is_done

class QLearningAgent:
    def __init__(self, env, alpha=0.1, gamma=0.9, epsilon=1.0,
    epsilon_decay=0.995, min_epsilon=0.01):
        self.env = env
        self.alpha = alpha
        self.gamma = gamma
        self.epsilon = epsilon
        self.epsilon_decay = epsilon_decay
        self.min_epsilon = min_epsilon

        # Initialize Q-table: Q[x, y, action] = 0.0
        self.q_table = np.zeros((env.grid_size, env.grid_size, len(env.
    actions)))

    def choose_action(self, state):
        """Epsilon-greedy action selection."""
        if np.random.rand() < self.epsilon:
            return np.random.choice(self.env.actions) # Explore
        else:
            x, y = state
            # Exploit: return the action with the max Q-value
            # Use random tie-breaking if multiple actions have the same
     max Q-value
            max_val = np.max(self.q_table[x, y])
            optimal_actions = np.where(self.q_table[x, y] == max_val)
    [0]
            return np.random.choice(optimal_actions)

    def update_q_value(self, state, action, reward, next_state):
        """The Bellman Update."""
        x, y = state
        nx, ny = next_state

        best_next_q = np.max(self.q_table[nx, ny])
```

```
101
102         # Q(S, A) <- Q(S, A) + alpha * [R + gamma * max_a(Q(S', a)) - Q
    (S, A)]
103         td_target = reward + self.gamma * best_next_q
104         td_error = td_target - self.q_table[x, y, action]
105         self.q_table[x, y, action] += self.alpha * td_error
106
107     def train(self, episodes=5000):
108         """Trains the agent over a specified number of episodes."""
109         rewards_history = []
110         for _ in range(episodes):
111             state = self.env.reset()
112             total_reward = 0
113             is_done = False
114
115             while not is_done:
116                 action = self.choose_action(state)
117                 next_state, reward, is_done = self.env.step(action)
118
119                 self.update_q_value(state, action, reward, next_state)
120
121                 state = next_state
122                 total_reward += reward
123
124             # Decay epsilon
125             self.epsilon = max(self.min_epsilon, self.epsilon * self.
    epsilon_decay)
126             rewards_history.append(total_reward)
127
128         return rewards_history
129
130     def get_optimal_policy(self):
131         """Extracts the optimal policy from the trained Q-table."""
132         policy = np.zeros((self.env.grid_size, self.env.grid_size),
    dtype=int)
133         for x in range(self.env.grid_size):
134             for y in range(self.env.grid_size):
135                 policy[x, y] = np.argmax(self.q_table[x, y])
136         return policy
137
138 def visualize_policy(env, policy):
139     """
140     Prints a visual representation of the policy grid.
141     Arrows represent the best action in each cell:
142     ^ (Up), > (Right), v (Down), < (Left).
143     Marks Start (S), Goal (G), and Pits (X).
144     """
145     # Mapping actions to symbols: 0: Up, 1: Right, 2: Down, 3: Left
146     action_symbols = {0: '^', 1: '>', 2: 'v', 3: '<'}
147
148     # We want to print top to bottom, so we iterate y in reverse order
149     for y in range(env.grid_size - 1, -1, -1):
150         row_str = ""
151         for x in range(env.grid_size):
152             state = (x, y)
153
154             if state == env.start_state:
```

```
                # Mark start state, but show the policy direction next to it
                symbol = f"S{action_symbols[policy[x, y]]}"
            elif state == env.goal_state:
                symbol = " G "
            elif state in env.pits:
                symbol = " X "
            else:
                symbol = f" {action_symbols[policy[x, y]]} "

            # Formatting to align columns
            row_str += f"[{symbol:^3}] "
        print(row_str)

# ========================================
# Example Execution
# ========================================
if __name__ == "__main__":
    print("--- Training with Slip Probability = 0.0 (Deterministic) ---")
    env_det = StochasticGridworld(slip_probability=0.0)
    agent_det = QLearningAgent(env_det, epsilon=1.0)
    agent_det.train(episodes=2000)
    policy_det = agent_det.get_optimal_policy()

    print("Optimal Policy (Deterministic):")
    visualize_policy(env_det, policy_det)

    print("\n--- Training with Slip Probability = 0.3 (Stochastic) ---")
    env_stoch = StochasticGridworld(slip_probability=0.3)
    agent_stoch = QLearningAgent(env_stoch, epsilon=1.0)
    agent_stoch.train(episodes=5000) # Takes a bit longer to converge with high noise
    policy_stoch = agent_stoch.get_optimal_policy()

    print("Optimal Policy (Stochastic - High Slip):")
    visualize_policy(env_stoch, policy_stoch)

    print("\nTraining complete. Q-tables populated.")
```

Code Walkthrough: Mechanics of the Q-Learning Agent

This section breaks down the implementation of the Q-Learning algorithm within the Stochastic Gridworld environment, detailing how the agent represents the world, transitions between states, and updates its knowledge over time.

21 Environment Representation (The MDP)

The environment, defined in the `StochasticGridworld` class, is a discrete representation of a Markov Decision Process (MDP).

States (S)
The state space consists of every valid (x, y) coordinate on a 5×5 grid.

- **Agent Representation:** The agent tracks its current position as a simple tuple: `self.current_state = (x, y)`.
- **Total States:** 25 distinct states.
- **Special States:** The Start state is $(0, 0)$. The Goal state is $(4, 4)$. The Pits are located at $(1, 1), (2, 1),$ and $(3, 1)$.

Actions (A)
At any given state, the agent has four possible discrete actions available to it:

- `0`: Up
- `1`: Right
- `2`: Down
- `3`: Left

Rewards (R)
The reward function is baked into the environment's `step()` method. The agent receives immediate feedback after every action:

- **Goal:** $+10.0$
- **Pit:** -10.0
- **Standard Step:** -0.1 (This small penalty encourages the agent to find the shortest safe path, rather than wandering endlessly).

21.1 The Transition Matrix (P) and Stochasticity

In a deterministic environment, taking action a in state s guarantees moving to the intended next state s'. However, our environment incorporates a `slip_prob` (e.g., 0.3 or 30%). This means the transition is stochastic, forming the core Markov property of the system.

While the code does not explicitly build a massive 25×25 transition matrix, the logic in the `step(action)` function mathematically functions exactly as one.

When the agent attempts to move (e.g., Action = `0` [Up]):

- **Intended Move ($1 -$ slip_prob):** With a 70% chance, the agent moves Up.
- **Slip Left (slip_prob$/2$):** With a 15% chance, the agent slips and moves Left.
- **Slip Right (slip_prob$/2$):** With a 15% chance, the agent slips and moves Right.

If the agent is at state $s = (1, 0)$ and chooses action $a = 0$ (Up), the implicit transition probabilities $P(s' \mid s, a)$ are:

- $P((1, 1) \mid (1, 0), \text{Up}) = 0.70$ (Moves into the pit!)
- $P((0, 0) \mid (1, 0), \text{Up}) = 0.15$ (Slips Left)
- $P((2, 0) \mid (1, 0), \text{Up}) = 0.15$ (Slips Right)

The agent does not know these probabilities beforehand; it must discover them through interaction.

21.2 The Learning Loop: Updating the Q-Table

The "brain" of the agent is the Q-table, initialized as a 3D NumPy array of zeros: `np.zeros((5, 5, 4))`. This table stores the expected cumulative return (Q-value) for taking a specific action in a specific state.

The ϵ-Greedy Strategy (Exploration vs. Exploitation)
In the `choose_action()` method, the agent decides its next move.

- It generates a random number. If the number is less than ϵ (epsilon), it selects a random action to explore the map.
- Otherwise, it exploits its current knowledge by looking at the Q-table for its current state and choosing the action with the highest value (`np.argmax(self.q_table[x, y])`).
- Epsilon decays over time, meaning the agent explores wildly at first, but slowly shifts to pure exploitation as its Q-table becomes more accurate.

The Bellman Update
After every step, the agent receives a tuple: `(state, action, reward, next_state)`. It uses this experience to update its Q-table via the Temporal Difference (TD) learning formula (the Bellman equation):

```
best_next_q = np.max(self.q_table[nx, ny])
td_target = reward + self.gamma * best_next_q
td_error = td_target - self.q_table[x, y, action]
self.q_table[x, y, action] += self.alpha * td_error
```

1. **Look Ahead (`best_next_q`):** The agent looks at the state it just landed in (`nx, ny`) and finds the maximum possible Q-value available from that state. This is its estimate of future rewards.
2. **Calculate Target (`td_target`):** It adds the immediate `reward` it just received to the discounted future rewards ($\gamma \times$ best_next_q). This forms the "target" value.
3. **Calculate Error (`td_error`):** It compares this new target value to its old estimate (`self.q_table[x, y, action]`).
4. **Update Table:** It adjusts its old estimate in the direction of the error, scaled by the learning rate (α).

Through thousands of episodes, the negative -10 rewards from falling into the pits propagate backward through the grid via this equation, lowering the Q-values of actions that lead near the pits, ultimately teaching the agent to avoid those states entirely.